

Module 1: Fish and heavy metal contaminants

updated 2024-7-29

Introduction

Members of the K'avi have fished the waters on their land for many years to provide food for their community. To manage these waters, K'avi tribal fishery managers have been monitoring fish growth rates across the local water bodies in their community.

However, many members of the community have begun to experience health problems associated with heavy metal poisoning. Historically, a factory used to be located upstream of the riverways fished by the K'avi, and waste from this factory was dumped into the nearby waters. Tribal fishery managers are now concerned that members of the community are being exposed to heavy metals through the fish that have been caught in these waters. As a result, they now want to start recording the concentrations of methylmercury in the belly fat of fish in the area. Health hazards depend on how much a person consumes as well as their size. The tribe uses guidance from the Environmental Protection Administration (EPA), who have determined a Reference Dose (RfD) of methylmercury at 0.1ug per kg of body weight. Additionally, the Food and Drug Administration recommends eating no more than 0.46ug methylmercury/g of fish for one serving per week (FDA (<https://www.fda.gov/food/environmental-contaminants-food/technical-information-development-fdaepa-advice-about-eating-fish-those-who-might-become-or-are>)). These values will be explored more in the code below.

Back in 1998, the tribal fisheries managers caught fish and measured their length and also the concentration of mercury in the fish. The goals of these managers was to learn how common it was for concentrations of mercury to be above the safety level of 0.46ug/g.

Once they collect these data, the tribal fishery managers will need to present their findings to members of the community so they can better understand the public health crisis at hand.

In this module, there are three main goals:

1. Explore data related to fish growth and heavy metal concentrations.
2. Explore K'avi community health data.
3. Connect these datasets and produce a graph that could be used to communicate the fishery managers' findings to their community.

NOTES TO SKC/ND DEVELOPERS AND INSTRUCTORS:

Learning goals for this module:

1. Students learn how R can be used as a calculator, and how values can be stored
2. Students become more comfortable opening R and loading in data from an external data file.
3. Students gain exposure to basic commands to explore their data before they begin to analyze it.
4. Students gain experience loading in packages (tidyverse and ggplot).
5. Students gain exposure to working with different data types(numeric, characters)

6. Students gain exposure to basic summary statistic arguments in R to pull the mean, median, max, and min values for different variables in the dataset.
7. Students become equipped with how to make a graph in R (tidyverse, ggplot) using variables pulled from a larger dataset. Can open a discussion point: how do we present data in a way that can help us understand the problem at hand? (i.e. in this case, it may be plotting mercury concentrations found in fish along different sites, and showing it next to a graph of mercury found in people across different sites)

END NOTE TO DEVELOPERS AND INSTRUCTORS

DAY 1: Getting started with fish mercury concentration data

Getting set up in Rstudio

Clearing Environment

The first thing we do anytime we start working in R is cleaning our environment. Your environment is where R stores all of the objects it is working with since the last time you cleared your environment. We clear the environment each time we open R so that there isn't anything still open from the last time you were using R that will mess up what we are trying to do this session. You can clear your environment with the following code or by clicking the broom icon that can normally be found on the top right section of RStudio.

```
rm(list = ls())
```

Setting the Working Directory

Note: if you are working out of a project, you do not need to run this section. You can check if you are in a project by looking in the top right corner of Rstudio for the project name, and in the bottom right panel you should see the files, including this one, listed.

In the following code we will tell the computer where we want it to look for files on our computer. This is called a working directory. We will discuss this more in depth later.

```
current_path = rstudioapi::getActiveDocumentContext()$path  
setwd(dirname(current_path))
```

Lesson 1.1: R basics and calculating screening values

Load in packages

"Packages" are computer code that someone wrote to do a particular thing. By loading a package, R downloads that code, so that you can do that thing yourself. Think of packages as tools for you to use in R that aren't automatically part of R when you first installed it. The package below allows you to do more things in R, and

makes coding much easier!

Below, we install the package “tidyverse” which we’ll use to organize data and to make pretty graphs. You only need to install a package once, but every time you use R you will have to load from the library using `library()`

```
# This line of code helps you install a package.
# You only need to do this once on your computer
if(!require("tidyverse")) install.packages("tidyverse")
```

```
## Loading required package: tidyverse
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.5
## ✓ forcats    1.0.0      ✓ stringr    1.5.1
## ✓ ggplot2    3.5.2      ✓ tibble     3.2.1
## ✓ lubridate  1.9.4      ✓ tidyr      1.3.1
## ✓ purrr      1.0.4
## — Conflicts — tidyverse_conflicts() —
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
# This loads a package that has already been installed, so you can use it.
library(tidyverse)
```

Before loading data, let’s think about the reference dose (RfD) of mercury. We can use R to calculate how much mercury can safely be consumed per day based on a person’s weight and fish consumption.

Creating objects in R

We will work with different types of objects and data types, including numbers, characters, vectors, and data frames. Here we start with creating objects for the variables and parameters used to calculate the screening value, or safe limit, of fish consumption. According to the FDA, the screening value, or amount of safe fish consumption, depends on a person’s body weight and how often they consume the fish.

First, the variables `weight` and `days_per_week` are set.

Once we create an object, we can use math or functions on it. Below we calculate `CR`, the mean daily consumption rate of the species of interest (g/d), based on the `days_per_week`

```
#-----
# change either of the variables for weight or consumption below!
#-----

# variable for weight, in pounds
weight = 160 # lbs

# avg daily consumption rate
days_per_week = 7


#-----
# Do not change anything below this line
#-----

# consumption rate based on days per week (110g is serving size)
cr = 110*days_per_week/7

print(cr)
```

```
## [1] 110
```

Calculating screening values for mercury in fish

Below we use the `weight` , mean daily consumption rate `cr` , and the reference dose `RfD` to calculate the screening value, based on FDA recommendations. Because the equation requires body weight in kg, we multiply our body weight in lbs by 0.45.

$$SV = RfD * weight * 0.45 / CR$$

```
# variable for RfD
RfD = 0.1 # ug mercury /kg body weight/day

# convert to RdD amount per day
# 1 lb = 0.45 kg
screening_value = RfD*weight*0.45/cr

# output screening value for serving size (ug mercury /day)
print(screening_value )
```

```
## [1] 0.06545455
```

The output above is the screening value (ug/g) for fish serving based on the inputs.

 Try changing the value for `weight` or `days_per_week` , and rerunning the two code chunks above to see the new safe screening value.

Creating vectors

Next we will create a vector, which is list of the same type of object, that can similarly be used in a function or equation. Below we have two vectors, `weight` and `servings`.

```
# run these lines one at a time or walk through together
```

```
# Create vectors
```

```
weight = c(140, 175, 200)
```

```
servings = c(1, 6, 3)
```

```
# can print entire vector
```

```
print(weight)
```

```
## [1] 140 175 200
```

```
print(servings)
```

```
## [1] 1 6 3
```

```
# or a single value from a vector
```

```
weight[1]
```

```
## [1] 140
```

Creating a data frame from vectors

Multiple vectors of the same length become columns in a data frame. Below we make a data frame named `sv_df`. When we preview it using the `head()` function, we can see it is comprised of two columns, with the same name as the vectors we created.

Inside a data frame, columns are referred to using `$`. We can also preview a single row, using brackets.

We then add a third column, where we apply the equation for the screening value to each row

```
# create data frame with two columns - from the vectors weight, servings
```

```
sv_df <- data.frame(weight, servings)
```

```
head(sv_df) # preview the entire data frame
```

```
##   weight servings
```

```
## 1    140        1
```

```
## 2    175        6
```

```
## 3    200        3
```

```
head(sv_df$weight) # see the column weight
```

```
## [1] 140 175 200
```

```
# Add column for screening value
```

```
sv_df$screen_val = RfD * sv_df$weight * 0.45 / (110 * sv_df$serving / 7)
```

```
head(sv_df)
```

```
##   weight servings screen_val
## 1    140         1 0.40090909
## 2    175         6 0.08352273
## 3    200         3 0.19090909
```

Visualizing screening value vs. servings per week with ggplot

Above, we used `head()` to preview our data frame. For longer data frames this will print out the first 6 rows.

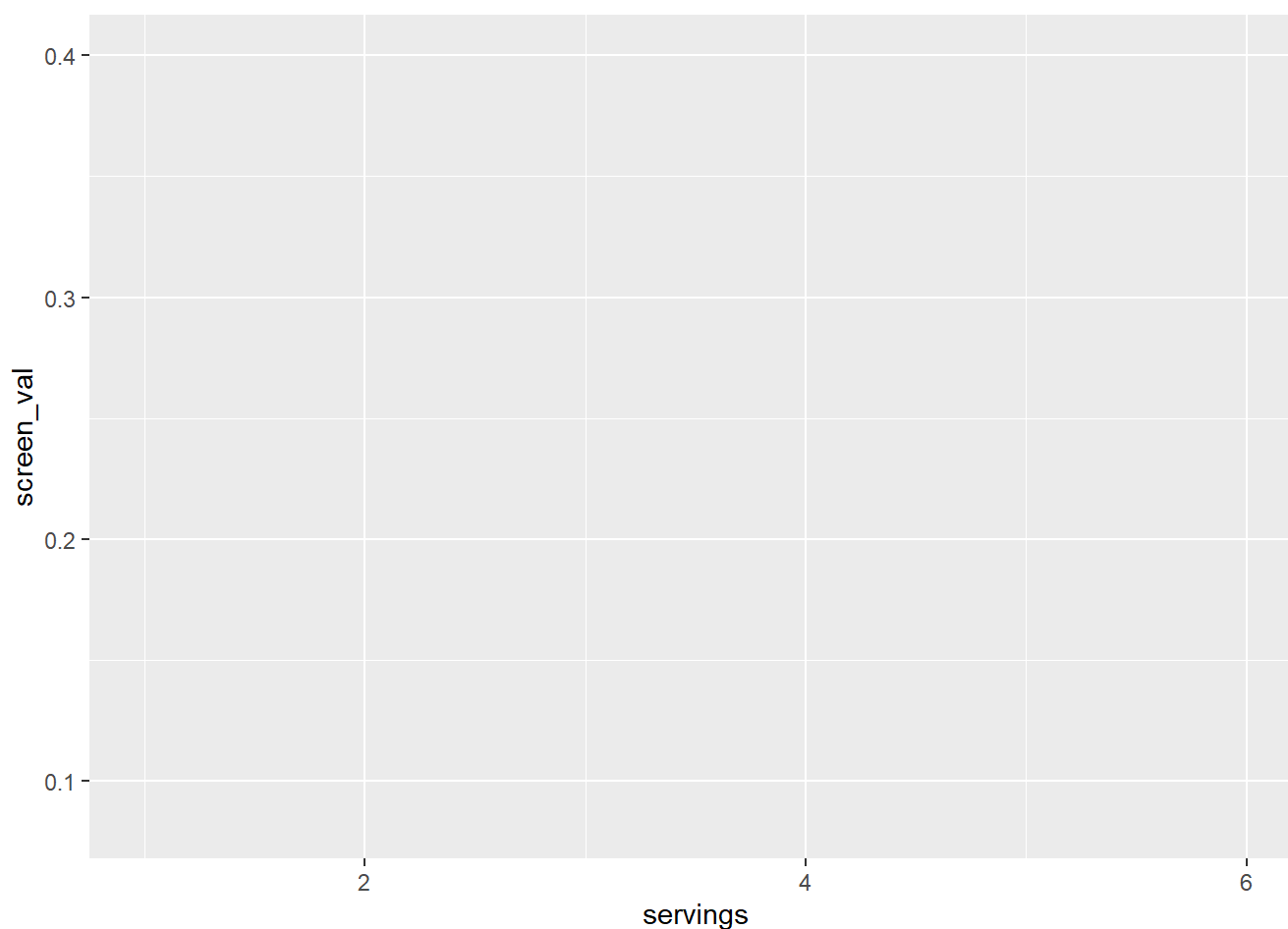
Another way to preview and explore data frames is to visualize it. Throughout this module, we will use `ggplot2` (<https://ggplot2.tidyverse.org/>). This package was loaded with `tidyverse`.

We will create a plot step by step, starting with running `ggplot()` with our dataframe as the input. We then add layers to it using a plus sign.

```
ggplot(data=sv_df)
```

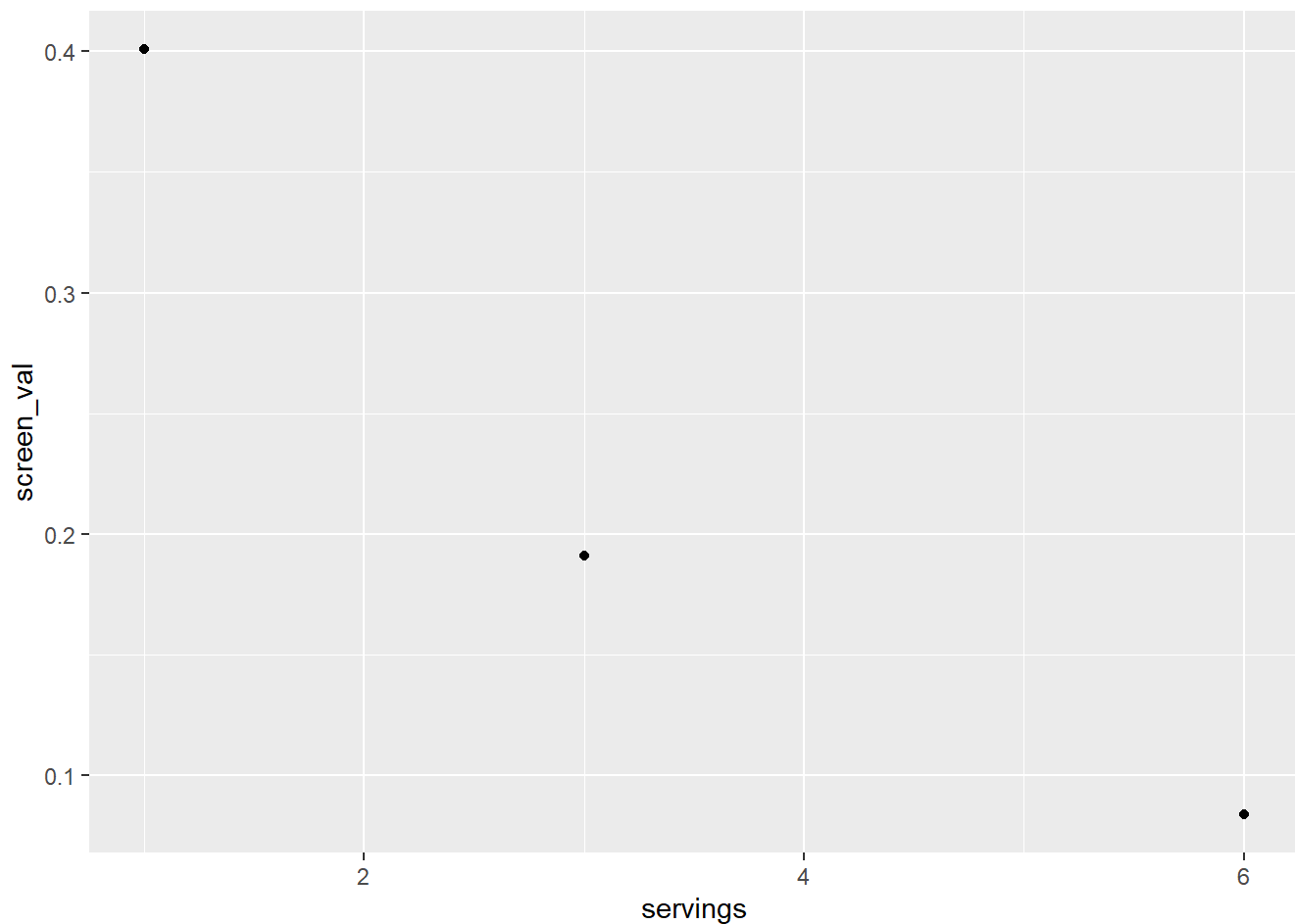
Starting with `ggplot()` created the plot, but we need to also include the variables we want to plot. We do this using `aes()` and then include the column name you want on the x-axis, and the column name for the y-axis.

```
ggplot(data=sv_df,  
       aes(x=servings, y=screen_val))
```



Next we need to assign a `geom` - this could be a scatter plot, line plot, or boxplot. We will start with plotting as columns, using `geom_col()`.

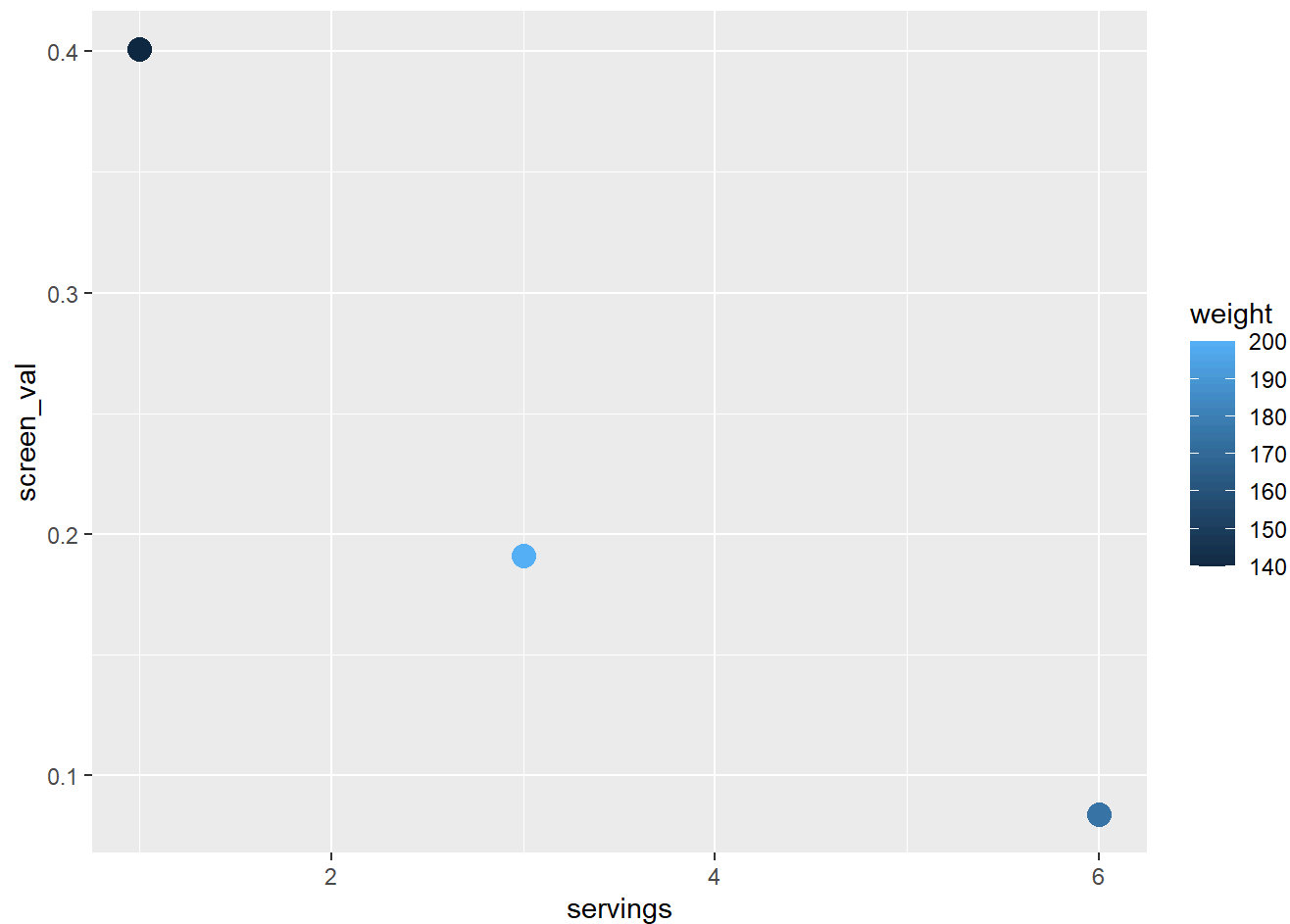
```
ggplot(data=sv_df,  
       aes(x=servings, y=screen_val)) +  
  geom_point()
```



Now we have a draft of our first plot! We can continue to add layers and change the settings to have the plot tell us more about our data, or to make it look nicer.

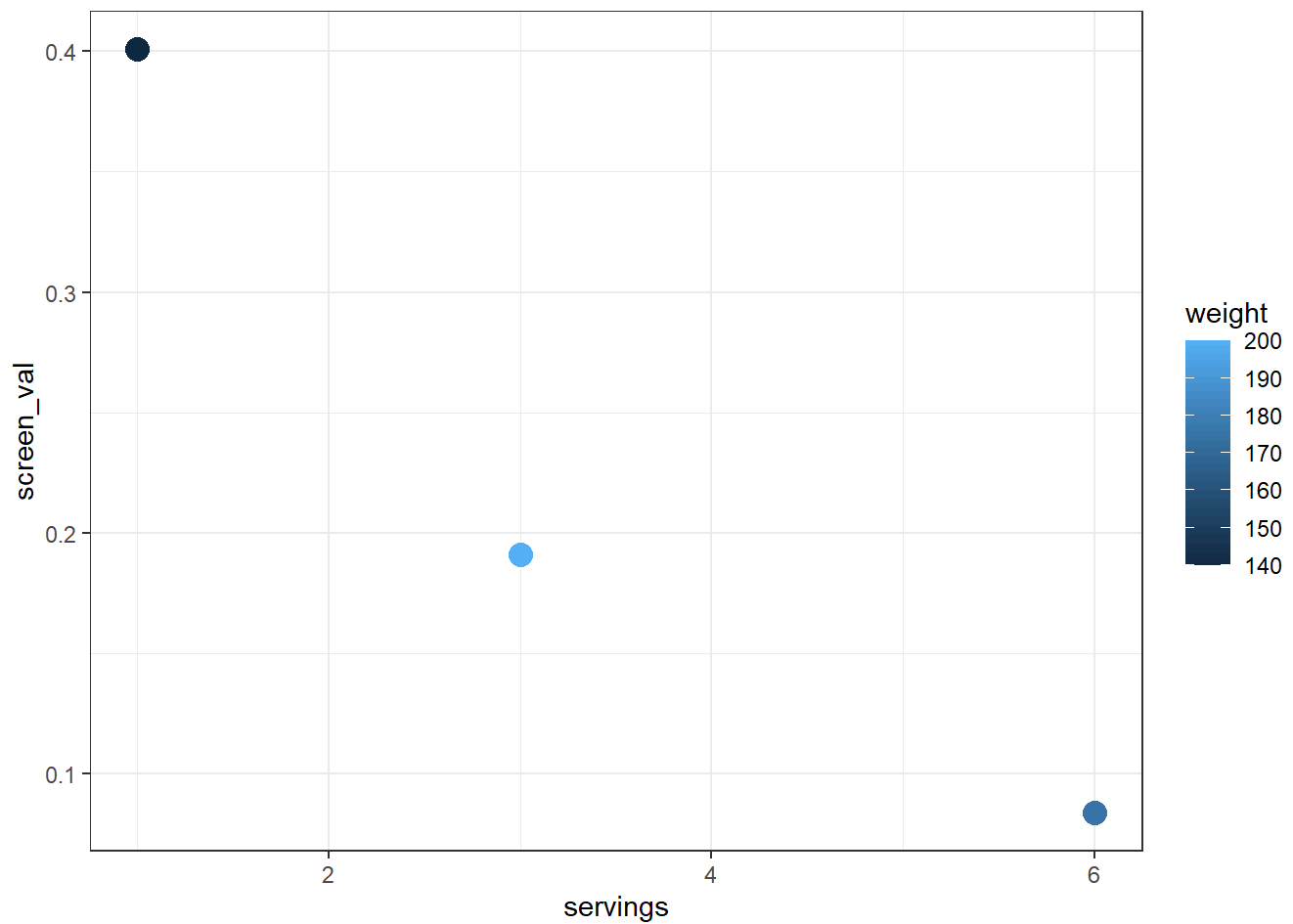
Variables can be included as colors - in this example we set the weight to fill which will fill in the column with a color gradient for our weight values.

```
ggplot(data=sv_df,  
       aes(x=servings, y=screen_val, col=weight)) +  
  geom_point(size=4)
```



We can also change the `theme`, which alters the background and style of our plot.

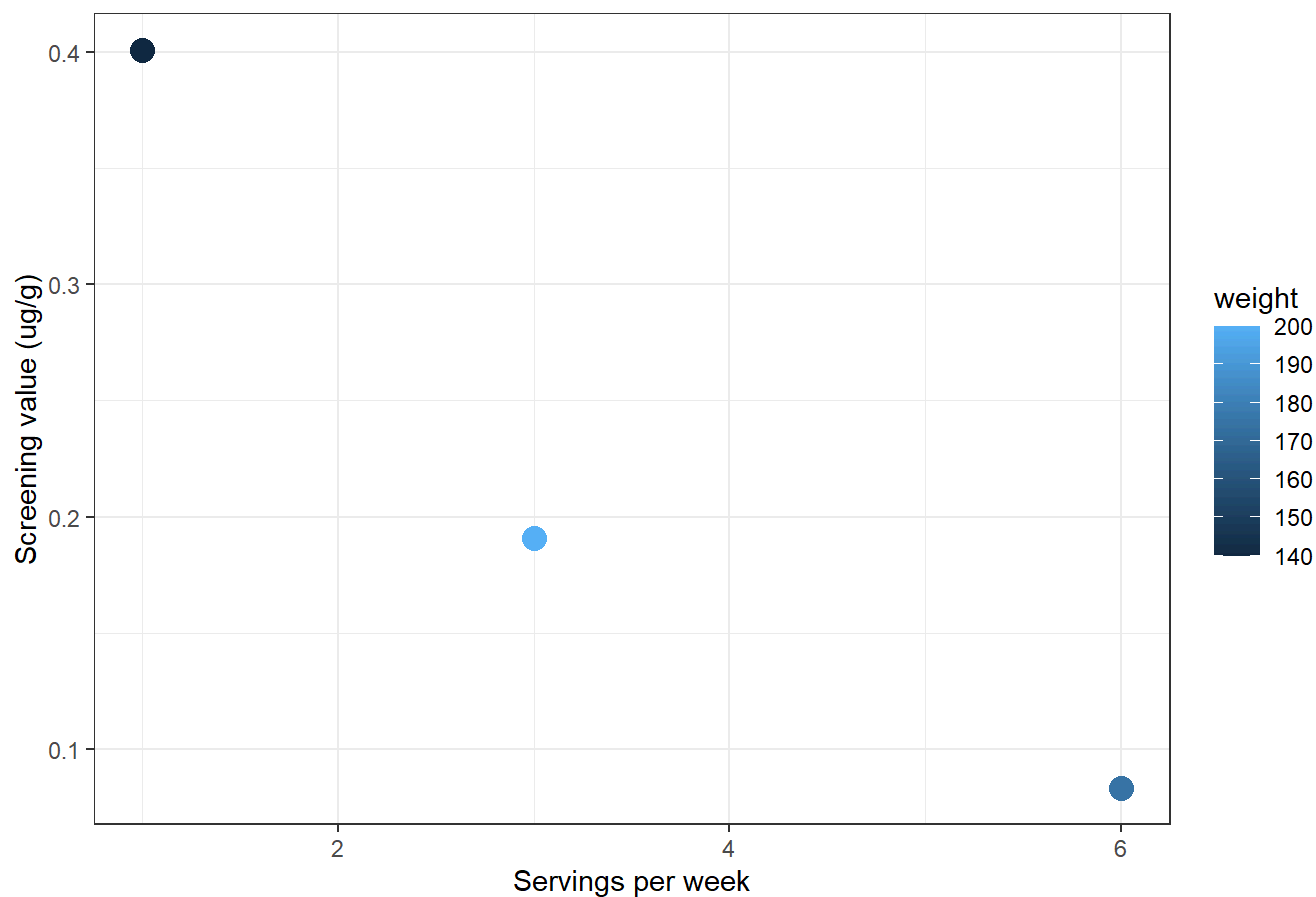
```
ggplot(data=sv_df,  
       aes(x=servings, y=screen_val, col=weight)) +  
  geom_point(size=4) +  
  theme_bw()
```



Finally we can add a title and change the labels to be more descriptive using `labs()` .

```
ggplot(data=sv_df,  
       aes(x=servings, y=screen_val, col=weight)) +  
  geom_point(size=4) +  
  theme_bw() +  
  labs(x="Servings per week", y="Screening value (ug/g)", title="Mercury Screening Value v. Servings per Week")
```

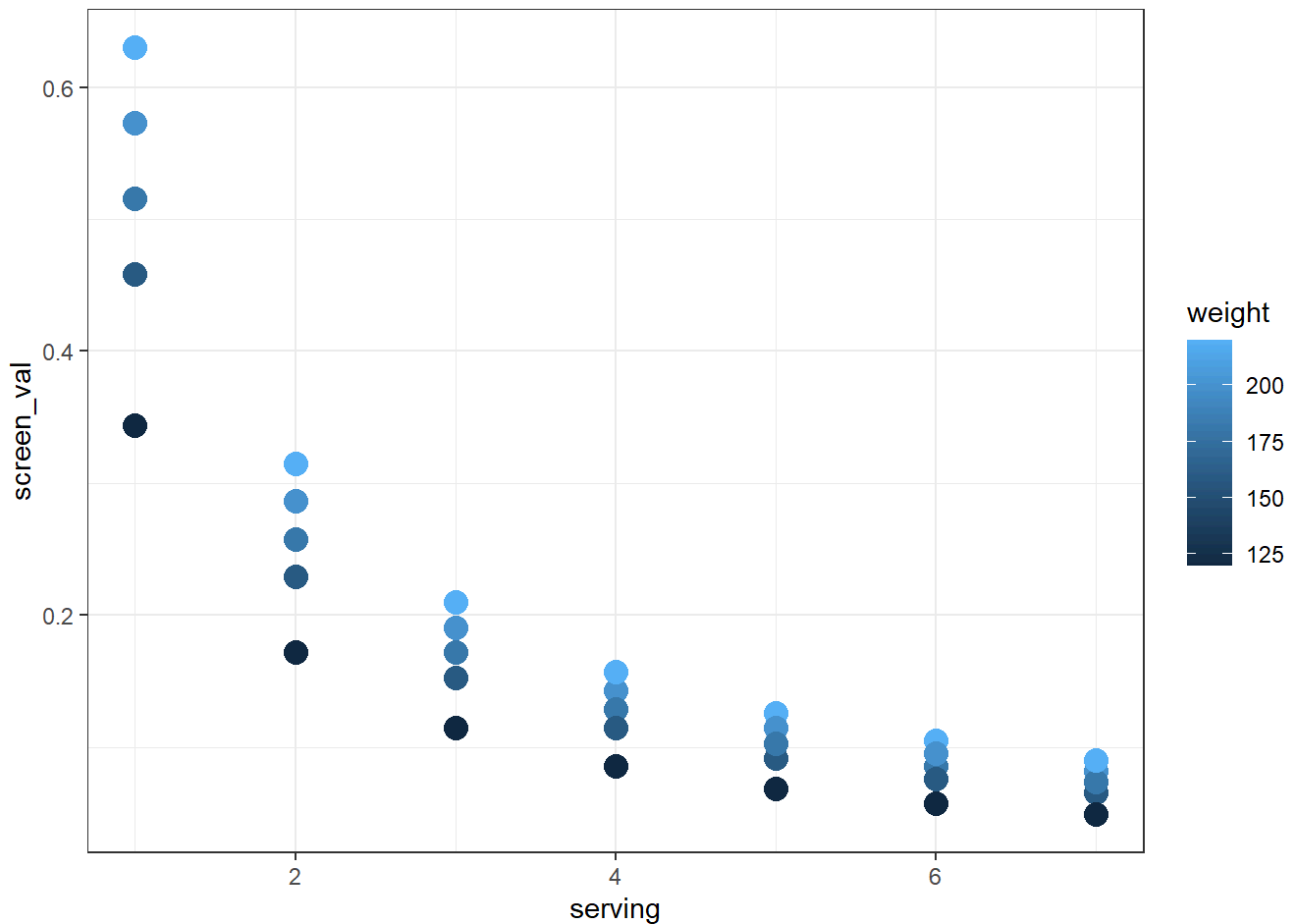
Mercury Screening Value v. Servings per Week



You should start to see a relationship between servings per week and screening value, but there are only 3 data points included above. Run the following code to create a similar plot with more data and get a better idea of the relationship between weight, servings per week, and screening value.

```
weight = rep(c(120, 160, 180, 200, 220), 7)
serving = rep(seq(1:7), 5)
sv_df_long = data.frame(weight, serving)
sv_df_long$screen_val = RfD*sv_df_long$weight*0.45/(110*sv_df_long$serving/7)

ggplot(sv_df_long) + geom_point(aes(x=serving, y=screen_val, col=weight), size=4) +
  theme_bw()
```



Class Questions

- How would you describe this graph and the relationship between screening value, consumption rate, and weight? What happens to the distribution in screening values as the number of servings per week increase?
- With this information in mind, consider how to make decisions with the data on mercury content in fish – how should we consider the screening value for different ages? For example, what screening value would be safer for children, who weigh less than the average adult?

Lesson 1.1 Recap

In this lesson we have learned to:

- define and print() an object in R
- use R for equations
- create a vector and a data frame
- visualize data with ggplot()
- describe the relationship between weight, servings of fish, and screening values of mercury

Lesson 1.2: Loading in and exploring the data

Now we will load in the dataset for the physical fish data

These data are the length and methyl mercury content of fish caught on the K'Avi reservation in 1998, soon after the polluting factory was shut down.

- Each row is data from one fish.
- The first column “length” is the length of the fish in cm.
- The second column “mercury” is the concentration of methylmercury (a poison) in the fish, in units of micrograms (ug) of methylmercury per gram of fish (g).

```
fish_1998 <- read.csv("fish_1998.csv", header = TRUE, sep = ',')
```

Let’s break down the different parts of what just happened:

1. We told R to read the data (a .csv file) using the command `read.csv()`
2. Within the (“”), we identify the data file from our working directory that we want to load into R
3. `header = TRUE` just tells R that the first row of data is the column names. The alternative is `header = FALSE`
4. `sep` is short for the word “separate”, and it tells R that each column in the csv is separated by a comma ‘,’
5. “fish” is the name we are giving that file in R, and is how we will refer to this data going forward.
6. `<-` is like `=` in a math equation, it tells R that “fish” is the same as the data “fish_1998.csv”
7. This command turns a file on your computer (fish_1998.csv) into a dataframe (fish_1998) that you can examine in RStudio. A ‘dataframe’ is the way R can see data that has rows and columns. When you run line of code above, R looks for the .csv file called “fish_1998.csv” and imports it into this RStudio session in a format (a database) that you can use for coding. The new dataframe (fish_1998) should appear in the RStudio “Environment” window in the upper right.

Previewing the data

Note: This will open a spreadsheet in new tab in this window (the “source editor” window).

- Look through these data to see if they make sense (any weird numbers? Any missing data?).
- Then close that tab and come back to this window.

```
View(fish_1998)
```

A shorthand way to view the data

If we don’t want to look at all of the data and instead just want to see what is going on in the first couple rows we can use the following code.

The command `head()` shows the column names and the first few rows of the data

```
head(fish_1998)
```

```
##   X   length  mercury
## 1 1 45.21643 0.4315457
## 2 2 44.34876 0.4969477
## 3 3 46.86563 0.4491865
## 4 4 41.67329 0.4137213
## 5 5 51.34514 0.7491135
## 6 6 56.66428 0.6807240
```

Class Questions:

- What are the columns in the R dataframe called fish_1998?
- What sorts of questions could the tribal fishery managers answer with this data?

Exploring the dimensions of the data

To answer these questions, it would be helpful to know how many fish have methyl mercury measurements. We can very quickly find that out with the following code.

The command `dim()` shows the dimension of the dataset. “Dimension” is the number of rows & number of columns.

```
dim(fish_1998) #shows the dimensions of the dataset, which will be important later on
```

```
## [1] 300 3
```

Class Question:

- How many fish did the managers measure mercury on in 1998?

Lesson 1.2 Recap

You now know the basic ways to look at and understand your data in R, which will come in handy any time you use R in the future. This included:

- Load a csv file as a data frame using `read.csv()`
- Preview the data frame using `head()`, `str()`
- Identify number of rows and columns using `dim()`

Lesson 1.3: Calculating descriptive statistics

Recall the maximum safe level of mercury to consume depends on how often fish are being consumed, and weight of person. According to the EPA, the maximum safe level of mercury to be consumed is around 0.46 ug/g/day if not frequently consuming fish.

Tribal fishery managers are interested in whether the mercury levels in local fish belly fat exceeds this amount. They are also interested in if the amount of mercury in fish is connected to fish size.

Below, we will analyze data points from the “fish” data set to try and answer these questions.

Descriptive statistics: mean v. median

What is the mean (average) amount of mercury found in local fish?

```
# Below, we will use "$" to select the column called "mercury" in the data
# If we wanted the mean fish length, we;d have written mean(fish_1998$length)

mean(fish_1998$mercury)
```

```
## [1] 0.4481177
```

The “mean” is the average level of mercury in the data.

Class Questions:

- Why would knowing the mean (average) amount of methylmercury in fish be useful for fishery managers and the community?

The “median” is the middle level of mercury data (half the fish in the dataset have more mercury and half have less).

- Means and medians are often similar, However:
 - They will be different if the data have a lot of large values or a lot of small valued (skewed data).
 - Sometimes you are asked to use one or the other, so it’s good to be careful about which you are calculating.

```
# remember, in the last code chunk, we used mean(fish_1998$mercury)
# The code below is similar, but it calculates the median.

median(fish_1998$mercury)
```

```
## [1] 0.4135396
```

Class Question:

- How would we interpret these median and mean values? Should the K’avi community be concerned?

Your turn

Work in groups to see if you can find these values using the commands `max()` and `min()`

For help, you can google “R min()” or you can type `?min()` in the “Console” window below, then read the help file in the “Output” window on the lower right.

```
# Type your code below to calculate the minimum and maximum values of methylemercury in
the 1998 fish dataset.
```

Class Question

- What are the minimum and maximum levels of mercury for fish in 1998?
- How would we interpret these values? Are they as important as the average?

Summarizing data

The mean, median, minimum and maximum are all “descriptive statistics.” This means that they provide key information about the distribution of data. Here, we want to know the distribution of mercury in fish collected in 1998.

The function `summary()` estimates all these descriptive statistics and more.

```
# The format of this summary() command is similar to other commands.
summary(fish_1998$mercury)
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
## 0.007764 0.278103 0.413540 0.448118 0.557474 1.894938
```

1st Qu. and 3rd Qu. refer to the 1st and 3rd “Quartiles”, which represent a quarter of the data. They tell you that 25% of the data is between that value, the Median, and either the Min. (1st Qu.) or Max. (3rd Qu.).

Class Question:

- Looking at all of these values, are there unsafe levels of mercury in local fish? Are there dangerous levels of mercury in most fish, or only some?

Lesson 1.3 Recap

In Lesson 1.3 we learned how to:

- get descriptive statistics for a vector
- interpret the summary statistics for our data set

How could we communicate these values?

However, summary stats can't answer every question, and as a list of numbers they would be very difficult to communicate to the K'avi community. It would be much more helpful to visualize the data in a graph, which we will do in the next part.

Lesson 1.4: Plotting mercury concentrations with ggplot

We want to first visualize the amount of mercury in all the fish, or a distribution of the mercury data.

We can do this by creating graphs!

If you run the code chunk below, it will make a graph of all the 1998 mercury data. We want to see the distributions of mercury in the fish to learn how common it is for fish to have mercury levels above the EPA's recommended safe level.

This code has a lot of details that we will work through later. For now, you can either just run the code chunk (by clicking the small green arrow in the upper right corner of the grey area), or you can explore the code line by line. We've provided some comments to help you out. This cheatsheet (https://images.datacamp.com/image/upload/v1666806657/Marketing/Blog/ggplot2_cheat_sheet.pdf) will also give you some tips.

Plot distribution of 1 variable

#For this plot, we want to see the distributions of mercury in the fish and how many of them are above the EPA's recommended levels

#ggplot is the function that we are using to create a plot. aes() is used to tell ggplot what we want to see on each axis, in this case x= 'fish' and y= mercury. x="fish" indicates that the x axis doesn't have a variable and is titled fish. This allows us to look at the variation in lengths of fish independent of another variable.

```
ggplot (fish_1998, aes(x= 'fish', y= mercury))+
```

#Since ggplot is just a function to create plots, we need to further specify what kind of plot it should create. We can do this by using the geom_jitter function, which will create the jitter plot seen below. We can also add a few more parameters to the graph, such as changing the width of the points (width = 0.1).

#The jitter plot is a variation on the scatterplot, but it separates the data so that we can distinguish how many points are above the unsafe mercury level. To see the difference, try geom_point() and then geom_jitter()

```
  geom_jitter(width = 0.1)+
```

#We can also add shapes to the plot. Here we added a line to show where the EPA's guideline for unsafe levels of mercury in fish would fall. To create this we used the geom_hline() function which simply creates a horizontal line at a y-intercept that you set. We can also adjust the color and appearance of the line with color = 'red'.

```
  geom_hline(yintercept = 0.46, linetype= "dashed", color= "red", size=1)+
```

#This allows us to relabel the x-axis

```
  xlab("fish")+
```

#This allows us to relabel the y-axis

```
  ylab("mercury (ug/g))+
```

#This allows us to title the graph

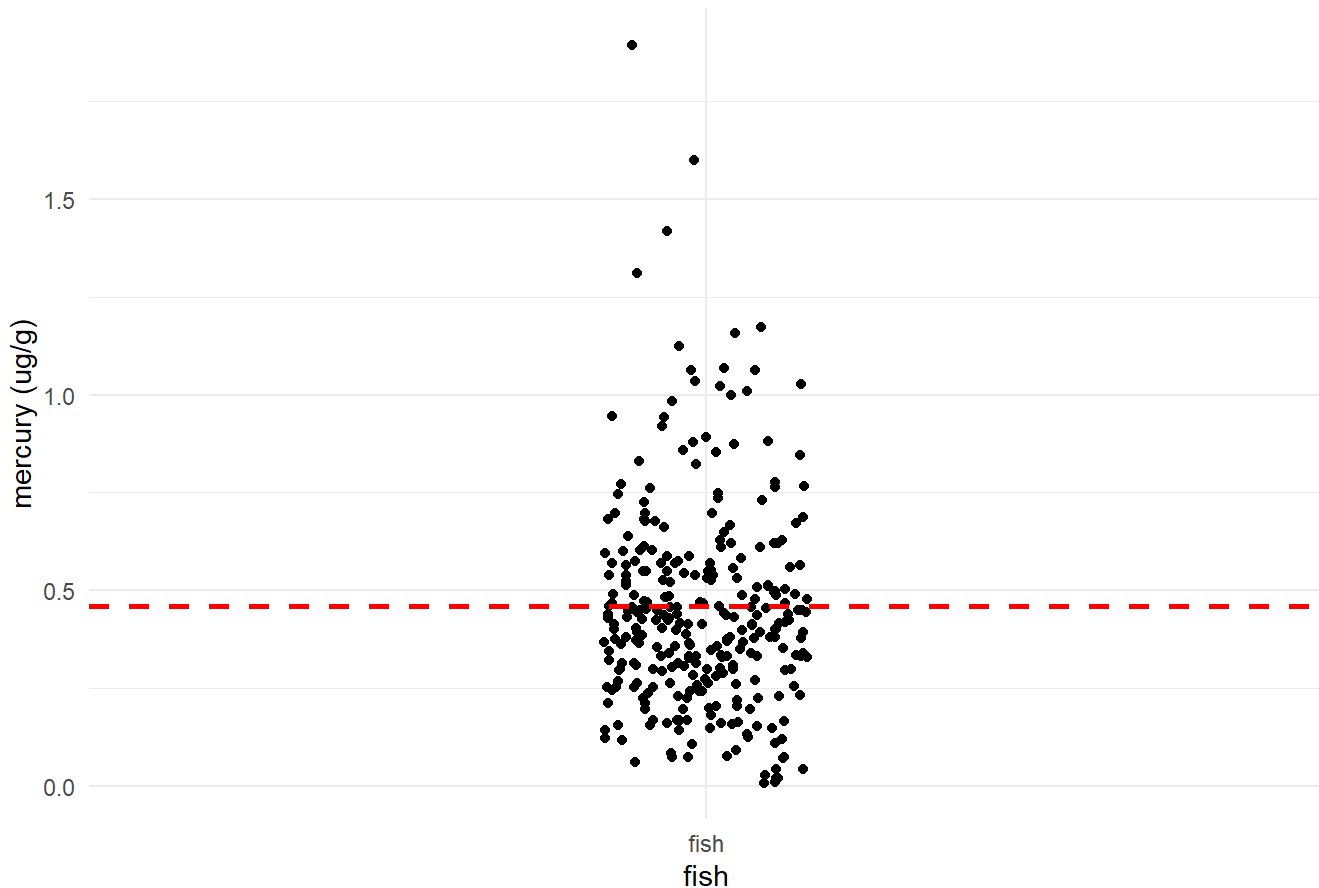
```
  ggtitle("1998 Mercury Distribution")+
```

#This adjusts the non-data components of the plot, so the titles, labels, background, and legends among other aspects.

```
  theme_minimal()
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

1998 Mercury Distribution



Class question:

- What does this distribution tell us about the mercury levels in the fish?

Boxplots: distribution of 1 variable

Next, Look back at the min, max, median, and mean values that were calculated above. Where would these values fall on this graph? Are the mean and median values above or below the level suggested by the EPA?

We can overlay a “boxplot” on the previous scatter plot. Boxplots are a way to show the mean, quartiles, minimum and maximum of a distribution. You can google “boxplot” to learn more.

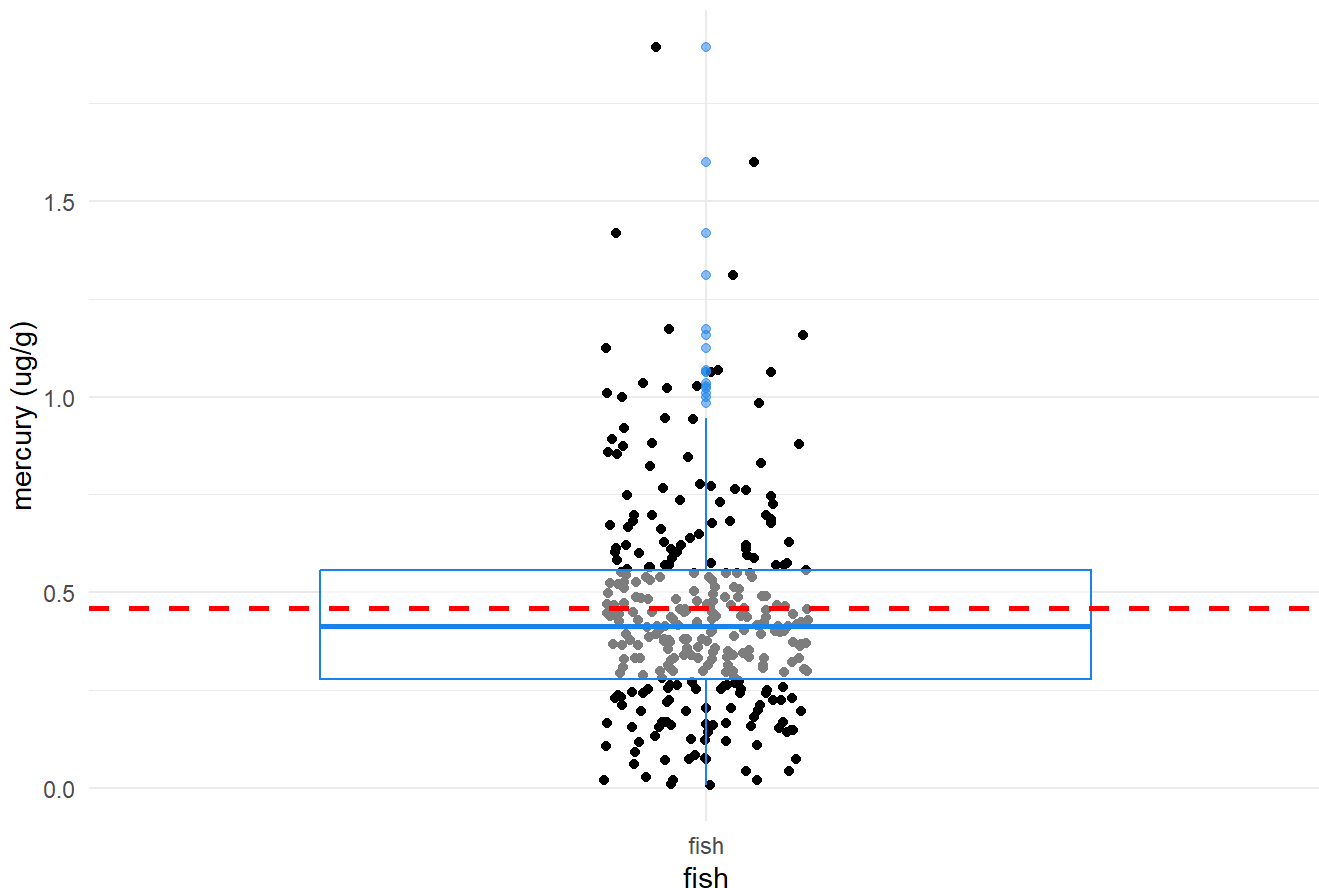
Like last time, you can just run this code if you want. We want to focus on the results. However, we’ve provided comments to help you understand each line of the code, if you want.

```

# These lines are almost the same as the last code chunk,
# but we added a '+' at the end, to add some new pieces to the graph.
ggplot (fish_1998, aes(x= 'fish', y= mercury))+
  geom_jitter(width = 0.1)+
  # We will overlay a boxplot on the jitter plot.
  # We can adjust its color as we did above,
  # We use alpha, to adjust the opacity of the boxplot.
  # This lets us see the jitter plot beneath it.
  geom_boxplot(alpha=0.5, color="dodgerblue2")+
  # As before, we add a line to represent the EPA's recommended safe mercury ingestion level
  geom_hline(yintercept = 0.46, linetype= "dashed", color= "red", size=1)+
  # The rest of the code is the same as the last code chunk:
  xlab("fish")+
  ylab("mercury (ug/g)")+
  ggtitle("1998 Mercury Distribution + Box Plot")+
  theme_minimal()

```

1998 Mercury Distribution + Box Plot



You have now seen what the mercury distribution in local fish looks like and connected it to the summary statistics that were calculated above!

Class Questions:

- From looking at this graph, are there unsafe levels of mercury in local fish? If so, is it in most fish, or only some? Did this graph tell you anything different than the summary statistics, or did it just tell you the same information in a different way?

- In 1998, was it safe for members of the K'avi community to eat fish from these waters?
- How comfortable would you have been in 1998 about eating a fish that you caught?

Scatterplots: Plot relationship between 2 variables

Mercury is known to build up in fish as they age: The oldest fish tend to have the highest levels of mercury. Older fish also tend to be bigger, so maybe if we know the relationship between fish size and fish mercury levels, that could give us useful guidance for which fish were safe to eat back in 1998.

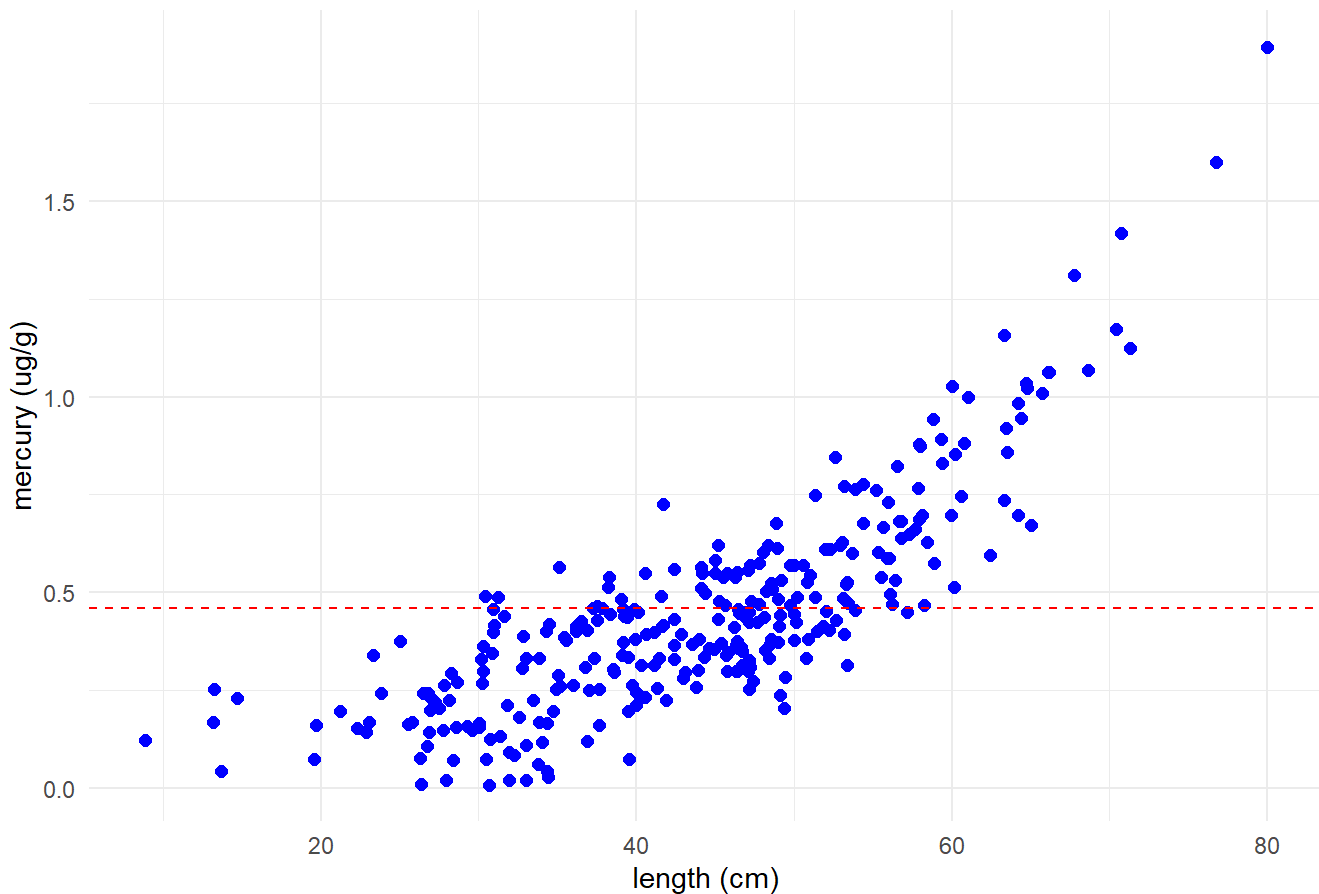
We'll make another graph of the same dataset. This time we'll make a scatterplot of both the mercury level and the length of each fish. A scatterplot shows the data as points in two dimensions (fish length and mercury content, here).

Again, you can just run the code (with the green arrow), but we've also provided comments to help you learn how it works. Our goal here is to let you see the kinds of graphs you can make in R and to help interpret the safety of the fish from 1998.

```
# This line defines a plot, called 'LengthMercury1998', as a ggplot().
# The ggplot() code is a little different from the single variable boxplot.
# Now, we are plotting two columns of the dataset:
# length will be on the (horizontal) x-axis
# mercury will be on the (vertical) y-axis
LengthMercury1998 <- ggplot(fish_1998, aes(x = length, y = mercury)) +
# geom_point() makes a scatterplot.
# We are going to plot the data points in blue
# and a little bigger than the default "size = 2"
  geom_point(color = 'blue', size = 2) +
# Now we're going to write labels for the two axes, and give the graph a title
  xlab("length (cm)") +
  ylab("mercury (ug/g)") +
  ggtitle("Mercury (ug/g) vs. Length (cm) (1998)") +
# As before, we will add a line to represent the EPA's recommended mercury level
# This time, we'll make a red dashed line.
  geom_hline(yintercept = 0.46, linetype = "dashed", color = 'red') +
# We'll use the same theme as last time.
  theme_minimal()

#Because we assigned the graph a label, we have to call that label
LengthMercury1998
```

Mercury (ug/g) vs. Length (cm) (1998)



Class Questions:

- What does this tell us about the relationship between fish length and mercury levels?
- Are there certain lengths of fish that have a higher amount of mercury?
- If you caught a fish in 1998, would this graph help you decide whether the fish was safe?

Lesson 1.4 Recap

In Lesson 1.4 we learned how to:

- plot the distribution and boxplot of a single variable, mercury
- plot a scatter plot of the relationship between two variables, mercury and fish length.

DAY 2: Did efforts to clean heavy metals improve the healthiness of fish?

Lesson 1.5: Putting it all together to explore the 2023 fish survey

The 1998 results showed health hazards from fish with high methylmercury motivated the tribe to undertake a project to remove contaminated sediments. In 2023, fifty years after the initial measurements, K'avi fisheries managers conducted another survey of mercury levels in fish. They carefully collected data from the same sites

they surveyed in 1998.

We will now download and analyze data collected from this second survey (fish_2023.csv).

Our goal is to see how successful the sediment removal efforts by the tribe were in reducing the amount of mercury present in the fish. We will do this by comparing our findings from the 1998 dataset to our current findings in the 2023 dataset.

Today's exercise allows you to apply the lessons you learned about coding in R from Day 1 to a new dataset.

Loading in packages and exploring the data

Like we did on Day 1 of the fish module, we first need to load in our datasets before we can explore them!

NOTE TO SKC/ND DEVELOPERS: When Day 2 starts, the class might need to take some preliminary steps before they get coding. We should consider:

- Setting the working directory (If we keep the data files in the same directory as this .Rmd file, they can skip this step. Or if you started a project, be sure to check the upper right corner of Rstudio to see that it is open.)
- Clearing the working directory
- These steps are described in the "Module 0" code.

Here is a commented out code chunk, to remind us to address this later...

```
# getwd()  
# setwd()  
# list.files()
```

END NOTE TO SKC/ND DEVELOPERS

Load in packages

We will be using the `tidyverse` packages as we used on Day 1. Since we've already installed this package, you just need to activate it for today using the `library()` command (see below)

Today, we're also using a new package called `patchwork`. This package allows you to plot graphs side-by-side. This will be useful later today, because we want to compare the mercury levels from 1998 side-by-side with those from 2023.

If you want to learn more about `patchwork` click this link: (<https://patchwork.data-imaginist.com/>)

Since we haven't installed `patchwork` yet, the code below installs it, then loads the library.

```
# This loads the "tidyverse" package that you've already installed.  
library(tidyverse)  
  
# This installs "patchwork"  
if(!require("patchwork")) install.packages("patchwork")
```

```
## Loading required package: patchwork
```

```
# This loads patchwork into the library, so you can use it in this program.  
library(patchwork)
```

Load the 2023 fish dataset

The dataset “fish_2023.csv” should be in the same working directory (folder) on your computer as this .Rmd file.

If that’s true, you can import these data into R using the same command that you used to load the 1998 data.

```
# This code brings the 2023 dataset into RStudio as a dataframe.  
fish_2023 <- read.csv("fish_2023.csv", header = TRUE, sep = ',')
```

We will repeat several of the steps we took to investigate the 1998 data with the new 2023, fish data. These steps are important because they allow you to check to see if the data loaded properly; to check to see if they make sense; and to start thinking about their implications for our scientific question, “Did the sediment cleanup efforts help the health of the fish and the community?”

We’ll start by viewing the data:

```
View(fish_2023)
```

Write your own code to view the top few lines of the data

Class Question:

- What would go in the parentheses of head() in order to get the output you want?

```
#Fill in the parentheses:  
head()
```

Class Questions

- What are the variables that this data set contains?
- In the code below, what would go in the parentheses of dim() in order to learn how many rows and columns the 2023 dataset has?

```
#Fill in the parentheses with the name of the dataframe for the 2023 fish data  
dim()
```

Descriptive statistics for the 2023 fish data

Now that we have successfully loaded in our data and explored its basic properties, we can now start to explore our data to learn how mercury levels in fish belly fat have changed from 1998 to 2023.

According to the EPA, the maximum safe level of mercury to be consumed is around 0.46 ug/g/day. Tribal fishery managers are interested in how mercury levels in fish belly fat have changed since 1998 to 2023, now that the rivers have been cleaned.

Below, we will analyze data points from the fish_2023 dataframe to answer these questions. We will start to do this by running some basic summary statistics on the mercury data from 2023.

Class Question:

- Work together to write code that tells you the descriptive statistics (median, mean, minimum, maximum, etc) for the mercury in the fish collected in 2023.

```
# write code for summary statistics here
```

Question for class:

- How do the mean and median values of mercury in 2023 compare to the EPA's standard?
- Do the descriptive statistics suggest that the mercury levels in fish have improved since 1998?

Recap: How could we communicate these values?

By running these summary statistics, we can see that mercury levels in fish caught in 2023 are lower than they were in fish that were caught in 1998, which is good news!

However, some fish in 2023 still have unsafe levels of methylmercury. Tribal fisheries managers know that people who catch these fish can't measure the mercury content, but they can measure the length of the fish they catch.

Data visualization

In the next section, we will make graphs that will show us if the length of fish that are safe to eat has changed since 1998. There are three goals for making these graphs:

1. You will learn a little more about making graphs using R.
2. Making these graphs will help us learn whether the sediment cleanup resulted in safer fish in 2023. The graphs let us look at all the data (not just the descriptive statistics).
2. You can use the graphs to decide what length of fish is safe for the community to eat. Then you can think about how to convey this information to the community.

Our goal is to show two plots next to each other. First we'll make the plot of the relationship between mercury and fish length in 1998 again. Then we'll make a very similar plot for the fish from 2023. Then we'll plot them side-by-side, which will let us answer the questions we have about how the healthiness of fish has changed over the 25 years since 1998.

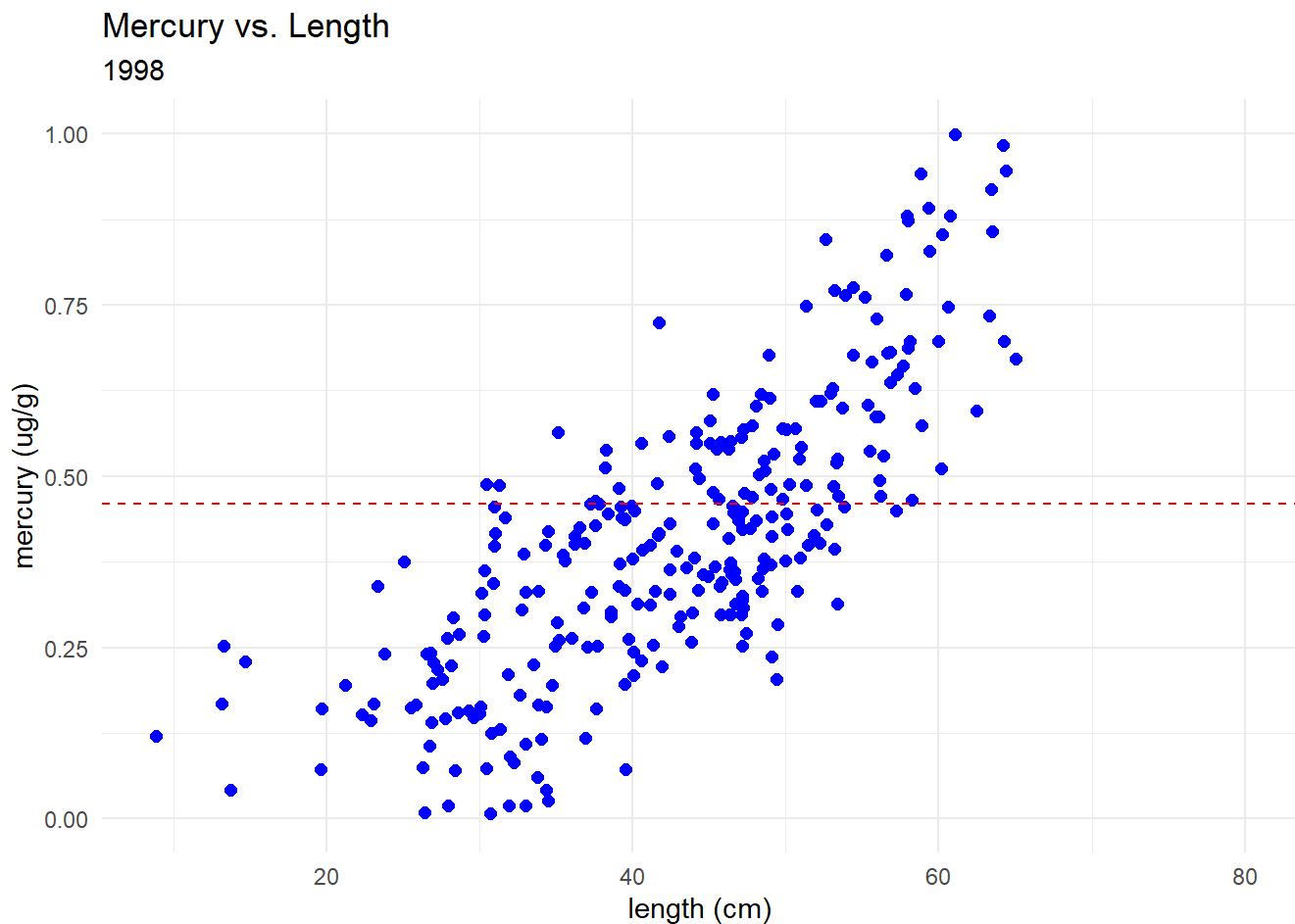
Our first step is to make a scatter plot of mercury and fish length from 1998. We're going to use the same code that we used before with one small change: We will set the range of the y-axis (the vertical axis), so that it will be the same as the y-axis of the 2023 data. That will help us compare the two graphs later.

Look more carefully at the code this time. You will see things that look similar to the code you've run before. This is a good starting point for learning to make graphs in R. To test yourself, see if you can describe what each line of code does. What makes sense? What seems confusing?

```
# This code is the same code we used before.
# If you look at each line, what makes sense? What is confusing?
plot1998<- ggplot(fish_1998, aes(x = length, y = mercury)) +
  geom_point(color = 'blue', size = 2) +
  xlab("length (cm)") +
  ylab("mercury (ug/g)") +
  ggtitle("Mercury vs. Length", subtitle="1998") +
  geom_hline(yintercept = 0.46, linetype = "dashed", color = 'red') +
# This is the only NEW line of code.
# We use the command ylim() to set the range of the y-axis
# The y-axis in this plot will start at 2 mg/kg mercury and end at 6 mg/kg
# We'll use the same range for the scatter plot of the 2023 data.
# That will make it easier to compare the fish across the two years.
  ylim(0,1) +
# We will specify the same "minimal" theme we used before.
  theme_minimal()

# Now let's plot the data from 1998
plot1998
```

```
## Warning: Removed 14 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Next, we'll make the same graph for the 2023 data. The code below is almost the same as the code you just saw. The only difference is that we make plot2023 using the 2023 fish data (in the last code chunk, we made plot1998 using the 1998 fish data).

This is a good opportunity to point out some **tips that will make you a better data scientist**:

- Feel free to use code that worked before. A nice thing about coding is that if you've done something once, you then have the code written down for future use.
- *However*, you have to be careful that you modify the code for the new situation. In this case, we have to change all the references to 1998 so that they refer to the 2023 data. We should also rename all the objects we make (like the graph that is called plot2023), so that we don't accidentally replace the old objects with the same name.
- It's really useful to look through the code line-by-line, so that you can catch all these little mistakes. You can even have a friend look at your code to see if it makes sense to someone else.
- If you follow this advice, things that seem difficult now become easier soon!

Here, we'll make our scatter plot for the 2023 fish length and mercury data:

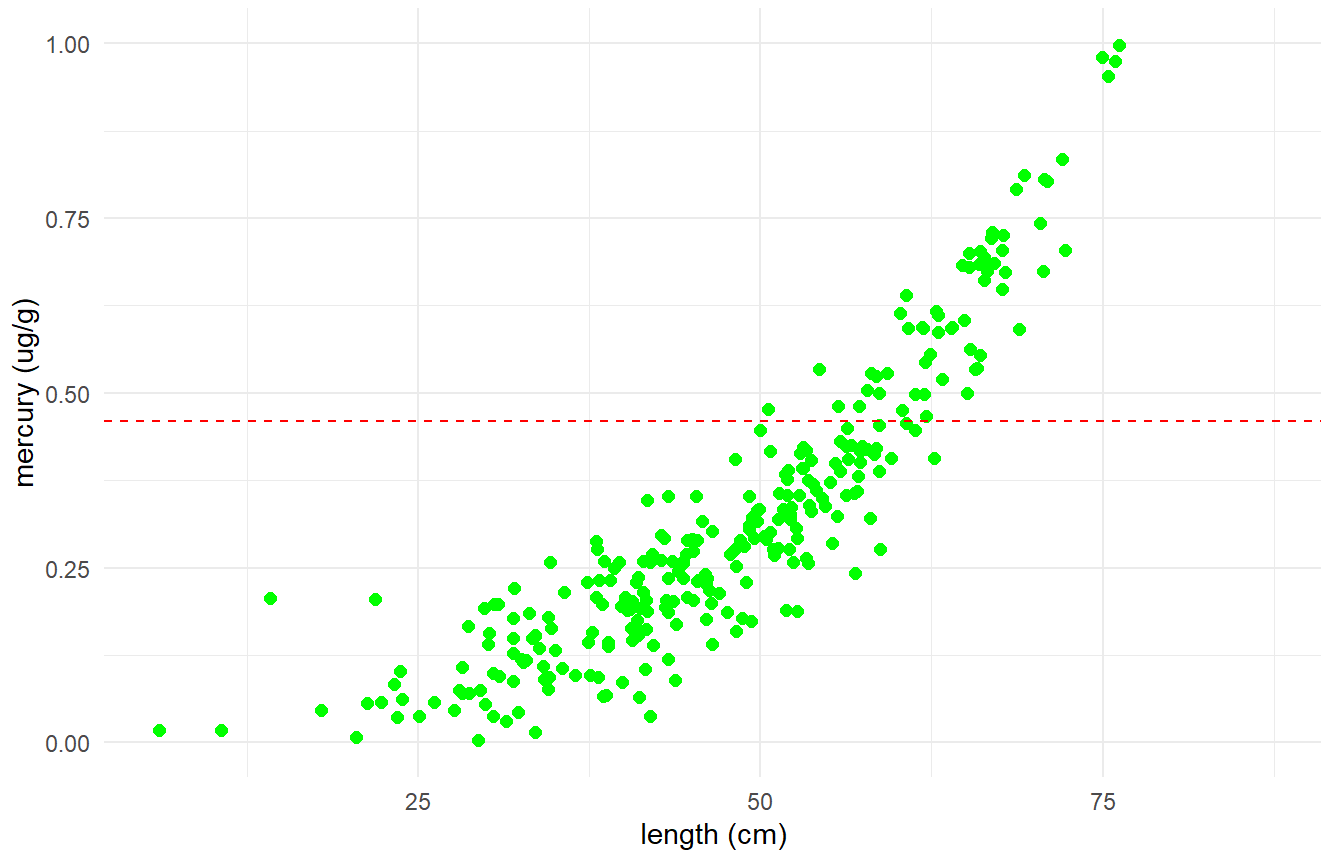
```
# See if you can understand the code below. What changes have been made?
plot2023 <- ggplot(fish_2023, aes(x = length, y = mercury)) +
  geom_point(color = 'green', size = 2) +
  xlab("length (cm)") +
  ylab("mercury (ug/g)") +
  ggtitle("Mercury vs. Length", subtitle="2023") +
  geom_hline(yintercept = 0.46, linetype="dashed", color = 'red') +
# We use ylim() to match the y-axis here to that of plot1998
  ylim(0, 1) +
  theme_minimal()

plot2023
```

```
## Warning: Removed 6 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

Mercury vs. Length

2023



The plot for the 2023 data looks pretty good. A side-by-side comparison with the 1998 data would help us see how much the healthiness of the fish has improved over the intervening 25 years.

Since we defined the two graphs as objects in R (`plot1998` and `plot2023`), and since we've loaded the package `patchwork` we can use the simple line of code below to plot the data from the two years side-by-side.

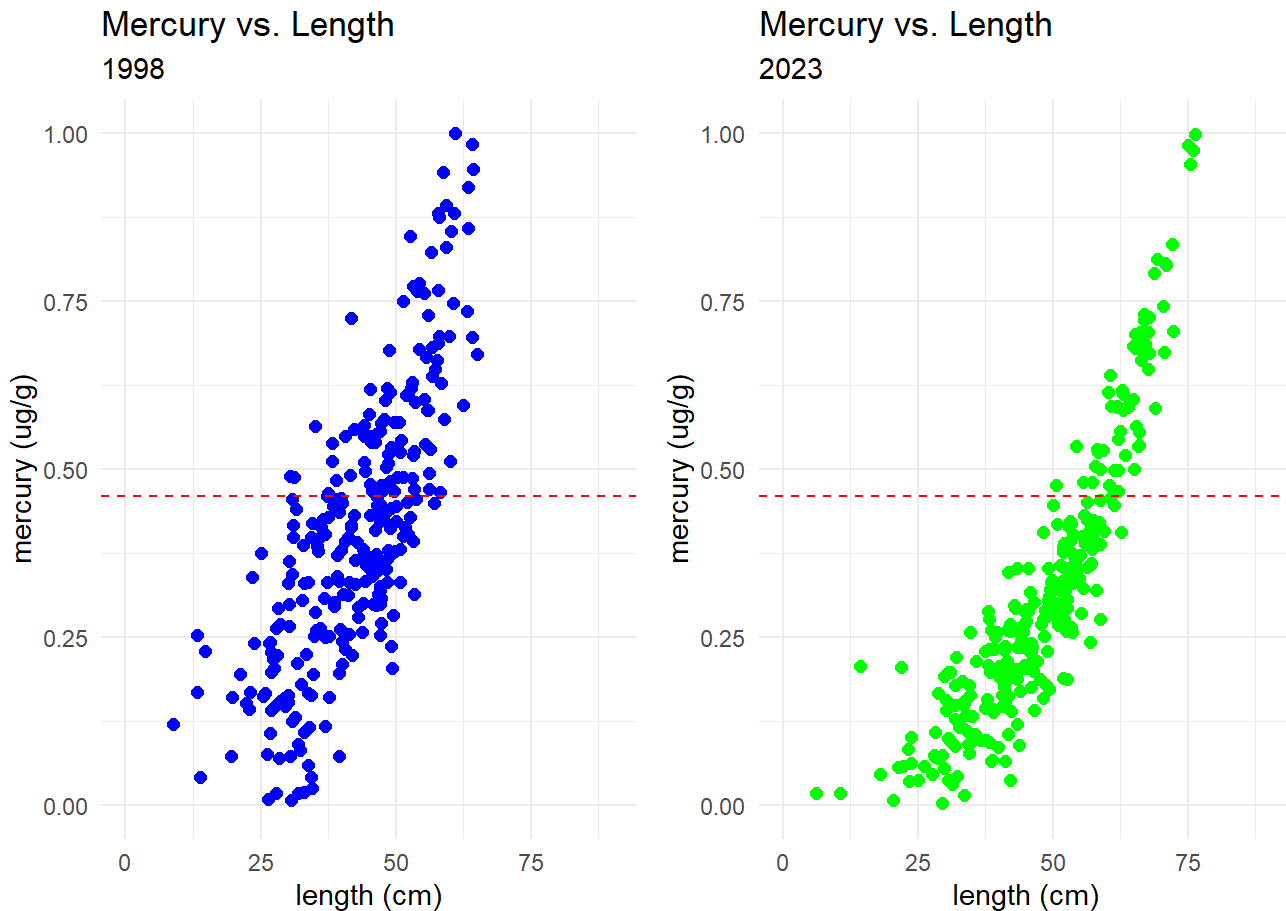
```
# plot from 1998 next to plot from 2023

# get scales to be the same
plot1998 = plot1998 + scale_x_continuous(limits=c(0,90))
plot2023 = plot2023 + scale_x_continuous(limits=c(0,90))

# we can use the patchwork package to be able to add plots
plot1998 + plot2023
```

```
## Warning: Removed 14 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

```
## Warning: Removed 6 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



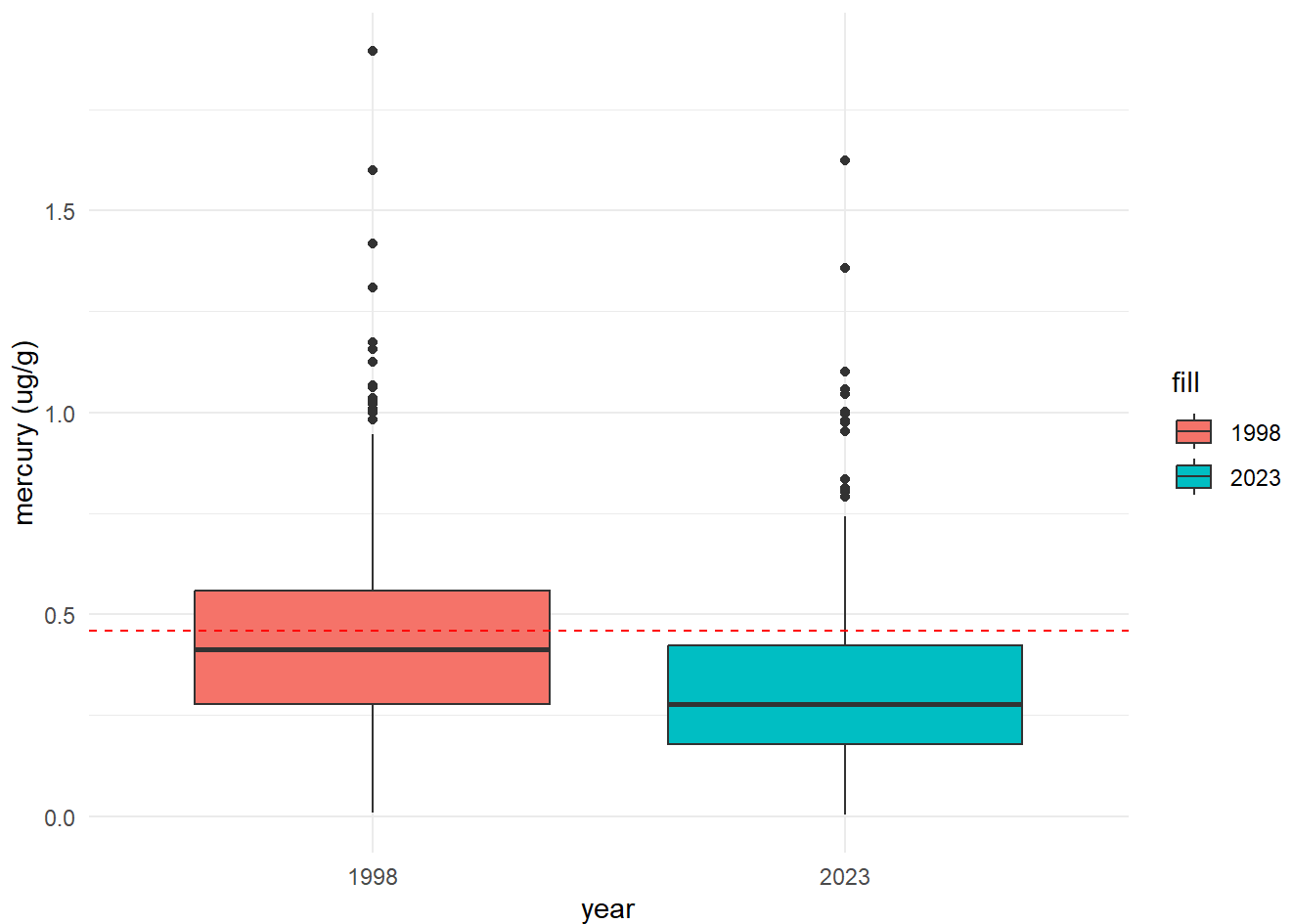
Class Questions:

- What is the safest large size fish to eat based on the 2023 dataset? How does this compare to the safest largest size fish that could be eaten based on the 1998 dataset?
- Based on these plots, do you think the efforts since 1998 to clean mercury from the sediments were successful?
- Work with a group to describe the evidence from these graphs for the success of the clean-up effort in a way that would be useful for the K'avi community.

Comparing groups in the same plot

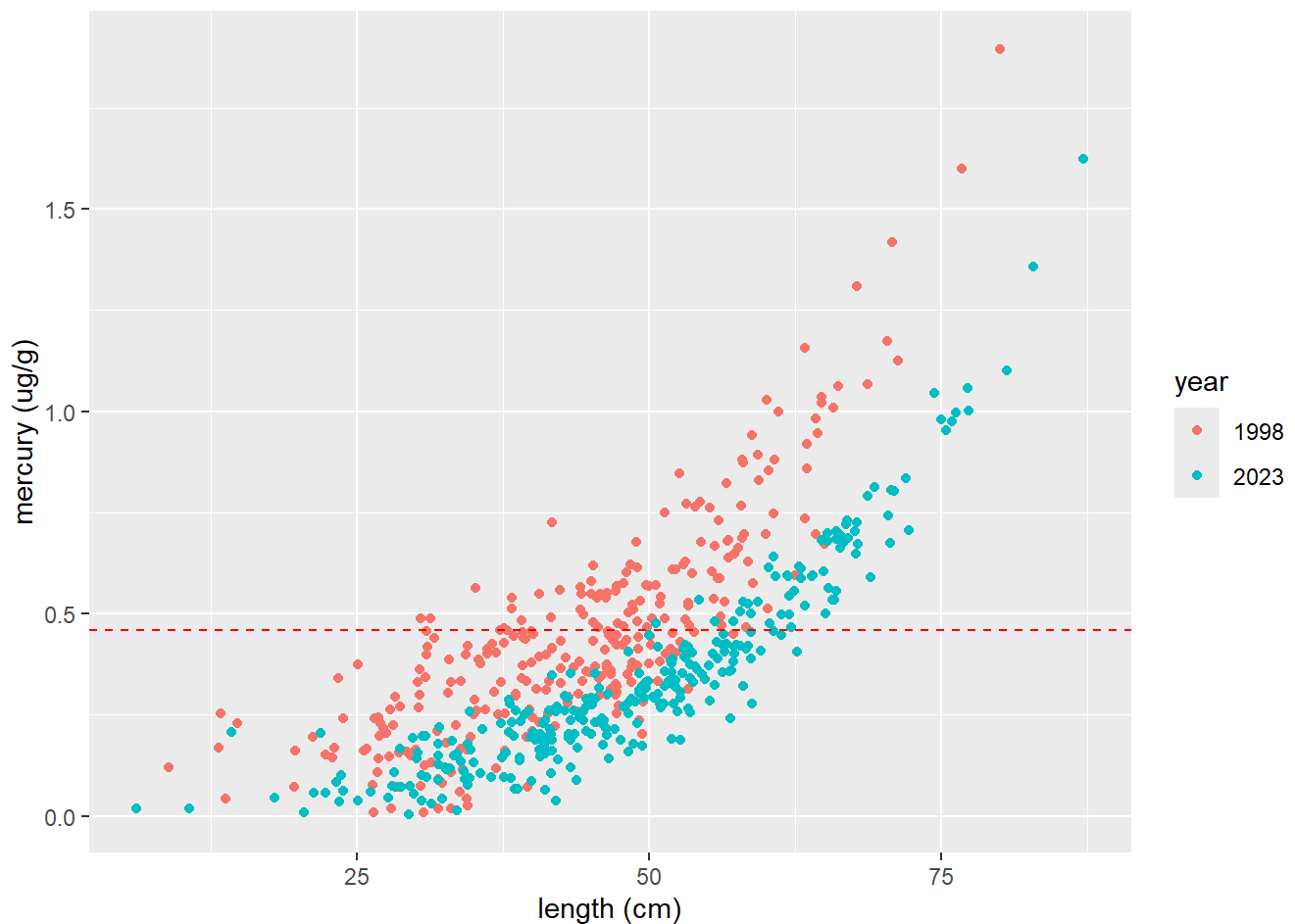
Using `ggplot()`, we can adjust the settings inside geoms to include both groups (in our case years) inside the same plot. Below are two different examples, a boxplot and a scatter plot with the year set to color. Look at the code below to see what is different.

```
# 1. boxplots
ggplot() +
  geom_boxplot(data=fish_1998, aes(x="1998", y=mercury, fill="1998")) +
  geom_boxplot(data=fish_2023, aes(x="2023", y=mercury, fill="2023")) +
  geom_hline(aes(yintercept=0.46), color="red", linetype="dashed") +
  labs(x="year", y="mercury (ug/g)") +
  theme_minimal()
```



2. scatter plot with color = year

```
ggplot() +
  geom_point(data=fish_1998, aes(x=length, y=mercury, color="1998")) +
  geom_point(data=fish_2023, aes(x=length, y=mercury, color="2023")) +
  geom_hline(aes(yintercept=0.46), color="red", linetype="dashed") +
  labs(color = "year", y="mercury (ug/g)", x="length (cm)")
```

**Class Questions:**

- How does the different ways of visualizing the data, as two separate plots and as different groups on the same plot, affect how we are able to interpret the data?
- Can you think of scenarios when it would be more useful to use one or the other?

Lesson 1.5 Recap

In Lesson 1.5, we have reviewed:

- Reading in and previewing a data frame of the 2023 fish survey
- Calculating descriptive statistics
- Data visualization with `ggplot()`
- Comparing different groups in data visualization